

Multi-Platform Container Rebuild Report

Multi-platform container rebuild report

Project: ExSOIL NSF Prototype **Period:** May-June 2026 (updated June 10) **Prepared by:** Steven Wangen, UW-Madison Data Science Institute

Section 1: Overview

What was rebuilt and why

This project runs the NCAR CTSM (Community Terrestrial Systems Model) land modeling framework inside a Docker container. Researchers pull the container image, launch it, and open Jupyter-Lab in their browser to run NEON tower site simulations and analysis notebooks. The container packages everything needed: an operating system, scientific Fortran/C libraries, the CTSM source code and build system, a full Python scientific stack (NumPy, xarray, cartopy, etc.), and Jupyter-Lab itself.

The original container image (`escomp/cesm-lab-neon`) was built in October 2022 on CentOS 8 (which lost security support in December 2021) and only supported Intel/AMD processors. It shipped Python 3.7 (end-of-life since June 2023) and hundreds of outdated packages. Most critically, it could not run natively on Apple Silicon Macs (M1/M2/M3/M4), which use a different processor architecture called ARM64. Team members on these machines had to run the container through a translation layer that was 2-3x slower and crashed frequently during computationally intensive operations like cartopy map rendering and model compilation.

Additionally, the NEON tower site workflow was non-functional. Investigation revealed that NEON support was never part of any CESM 2.x release; the original container used an undocumented custom build that grafted NEON support from a development branch. This graft broke as the components diverged.

The container was rebuilt from scratch on Ubuntu 24.04 with standalone CTSM 5.4.043, native support for both processor architectures, modern packages, and a reproducible build system.

What multi-platform support means

A Docker image is compiled code, and compiled code is specific to a processor architecture. An image built for Intel/AMD (called “amd64” or “x86_64”) contains machine instructions that ARM processors cannot execute directly, and vice versa. When you run an amd64 image on an ARM Mac, Docker uses a technology called QEMU emulation to translate instructions on the fly. This works, but it is slow, uses more memory, and can trigger subtle bugs in complex software.

Multi-platform support means publishing a single image tag (like `latest`) that actually contains two architecture-specific images behind the scenes, organized in what Docker calls a “manifest list.” When someone runs `docker pull`, their Docker client automatically selects the correct image for their machine. An engineer on an Intel Linux server gets the amd64 image. An engineer on an M3 MacBook gets the arm64 image. Neither needs to know or care about the other architecture.

The approach

Rather than cross-compiling the old image, the team rebuilt the entire stack on a new foundation. The original image compiled five scientific libraries (MPICH, HDF5, NetCDF-C, NetCDF-Fortran, PNetCDF) from Fortran and C source code inside the container, a process that took 30-45 minutes and was architecture-specific. The rebuild replaced all of this with pre-built binary packages from conda-forge, a community package repository that publishes builds for both amd64 and arm64. This reduced build time to about five minutes and eliminated the most complex source of architecture-dependent code.

The project also migrated from the full CESM 2.2.2 (which lacked native NEON support) to standalone CTSM 5.4.043 (which includes the NEON workflow natively), trimmed the Python environment from approximately 400 packages to 35 direct dependencies, introduced reproducible lockfiles for both architectures, and created a three-tier automated test suite that validates the container on each platform.

What changed for downstream consumers

Nothing, operationally. The image is published to the same GitHub Container Registry address. `docker pull` and `docker run` commands are unchanged. On Apple Silicon Macs, the container now runs natively instead of through emulation, so it is faster, more stable, and does not crash during heavy computation. The container also now supports end-to-end NEON simulations (create case, build model, run, archive, analyze) which was not possible with the previous image.

Section 2: Technical detail

Base image selection

Decision: `ubuntu:24.04` (untagged digest, pulled at build time).

Rationale (documented in ADR-0001): The upstream image used CentOS 8 (EOL). Six alternatives were evaluated:

Option	Multi-arch	Compressed size	EOL	Community fit
Ubuntu 24.04	amd64, arm64 + 4 more	~29 MB	Apr 2029 (free), 2036 (ESM)	Pangeo, Jupyter Stacks
AlmaLinux 9	amd64, arm64	~30 MB	May 2032	CERN, Fermilab

Option	Multi-arch	Compressed size	EOL	Community fit
Rocky Linux 9	amd64, arm64	~44 MB	May 2032	CIQ HPC
Debian 12	amd64, arm64	~47 MB	Jun 2028 (LTS)	Less common for HPC
jupyter/scipy-notebook	amd64, arm64	~1.5 GB	Follows Ubuntu	Jupyter community
condaforge/miniforge3	amd64, arm64	~148 MB	Follows Ubuntu	Pangeo pattern

Ubuntu was selected because both Pangeo and Jupyter Docker Stacks have standardized on it, it has the longest free support window, and its `apt` ecosystem has the best-documented path for Fortran/C scientific library compilation.

Dockerfile structure

The Dockerfile uses a three-stage build, all stages deriving from the first:

Stage 1 ("base")	FROM ubuntu:24.04	Conda environment + scientific stack
Stage 2 ("ctsm")	FROM base	CTSM source + machine config
Stage 3 ("app")	FROM ctsm	Project notebooks + analysis modules

This is a linear chain, not a parallel multi-stage build. There is no separate builder stage; compilation tools (gfortran, gcc, cmake) remain in the final image because CTSM's `case.build` workflow requires them at runtime.

Stage 1: base

1. `apt-get` installs system packages: compilers (build-essential, gfortran), build tools (cmake, m4, make), version control (git, subversion), Perl XML processing (perl, libxml-libxml-perl), and utilities.
2. Miniforge installer detects architecture automatically:

```
RUN wget -qO /tmp/miniforge.sh \
    "https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-$(uname -m).sh"
```

3. Conda environment installed from explicit lockfiles with architecture selection at build time:

```
RUN ARCH=$(uname -m) \
    && if [ "$ARCH" = "aarch64" ]; then LOCKFILE=/tmp/conda-linux-aarch64.lock; \
    else LOCKFILE=/tmp/conda-linux-64.lock; fi \
    && mamba install --name base --file "$LOCKFILE" --yes --quiet
```

4. Optional Dask layer controlled by build arg (INSTALL_DASK_DISTRIBUTED=false).

Stage 2: ctsm

1. Clones CTSM 5.4.043 and runs `git-fleximod update` to pull component repositories (CLM, CIME, CDEPS, CMEPS, MOSART, PIO).

2. Overlays container machine configs with conda-forge library paths (`$CONDA_PREFIX` instead of `/usr/local`).
3. Patches the NEON usermods to remove `MPILIB=mpi-serial` (conflicts with conda-forge MPICH at runtime).
4. `chown -R user:ctsm` transfers ownership of the CTSM tree to the runtime user (required because CIME creates build artifacts in-tree).

Stage 3: app Copies project-specific code: analytics modules, `run_neon_v2` wrapper, notebooks. Sets `PYTHONPATH` to include CTSM Python module directories. Configures git user for CIME (which commits during `case.build`).

Cross-compilation strategy

There is no cross-compilation. Both architectures build natively (amd64 on native hardware, arm64 under QEMU emulation on the CI runner). This is viable because:

- **No source compilation of Fortran/C libraries in the image build.** All scientific libraries come from conda-forge pre-built binaries.
- **The Miniforge installer and conda-forge lockfiles are architecture-specific.** Each architecture gets its own resolved dependency graph.
- **CTSM Fortran compilation happens at runtime (`case.build`), not during the Docker build.** The image ships source code and a toolchain; the user compiles when they create a case. Tested and confirmed working on native arm64 in ~100 seconds.

Dependency management and reproducibility

The project uses a three-file system documented in ADR-0003:

1. **environment.yml** (human-maintained): ~35 direct dependencies with loose version constraints. Python ≥ 3.11 (no upper bound; CTSM 5.4's CIME 6.1 supports Python 3.12+). cmake with no upper bound (CTSM's PIO 2.6 is compatible with cmake 4.x).
2. **conda-lock.yml** (generated): Unified lockfile produced by `conda-lock lock` for both platforms. Contains exact versions and hashes.
3. **conda-linux-{64,aarch64}.lock** (generated): Explicit lockfiles rendered from `conda-lock.yml` via `conda-lock render`.

CTSM build compatibility fixes

The migration from CESM 2.2.2 to standalone CTSM 5.4.043 resolved most of the original compatibility issues. The remaining fixes are in the container machine config:

Issue	Root cause	Fix	Location
<code>rdtsc</code> asm error in GPTL	x86-only instruction (<code>HAVE_NANOTIME</code>)	Removed <code>-DHAVE_NANOTIME</code> <code>-DBIT64</code> from GPTL <code>CPPDEFS</code>	<code>container.cmake</code>

Issue	Root cause	Fix	Location
MPI type mismatch	GCC 10+ strict type checking	<code>-fallow-argument-truncation</code>	<code>environment.cmake</code>
BOZ hex literal error	GCC 10+ strict BOZ checking	<code>-fallow-invalid-code</code>	<code>container.cmake</code>
<code>_FillValue</code> macro renamed	NetCDF-C renamed to <code>NC_FillValue</code>	<code>-D_FillValue=NC_FillValue</code>	<code>Containerfile.cmake</code>
ESMFMKFILE not set	NUOPC coupling requires ESMF	Set in <code>config_machines.xml</code> environment	
MPILIB=mpi-serial conflict	Conda-forge MPICH on library path conflicts with CIME serial stubs	Patched NEON usermods to remove override	Dockerfile

Issues resolved by the CTSM migration (no longer apply): PIO2 source patches, Python 3.12 pin, cmake <4 pin, bundled `six.py` conflicts, `import imp` deprecation.

Testing strategy

A three-tier pytest framework runs inside the container via `tests/run_container_tests.sh`:

```
./tests/run_container_tests.sh           # all tiers
./tests/run_container_tests.sh tier0      # smoke only (~4s)
./tests/run_container_tests.sh tier0 tier1 # smoke + case creation (~6s)
./tests/run_container_tests.sh tier2      # includes case.build (~100s)
```

Tier	Tests	Runtime	What it validates
0	63	~4s	Python imports (28 packages), CIME/CTSM module imports, gfortran/gcc/mpiexec presence, Fortran+MPI compilation, NetCDF library presence, CTSM install integrity, env vars
1	13	~3s	<code>query_config --compsets/--grids</code> , <code>create_newcase</code> , <code>case.setup</code> , <code>xmlchange/xmlquery</code> round-trip, <code>run_tower --help</code> , NEON site listing
2	14	~100s	Full <code>case.build</code> producing <code>cesm.exe</code> (~100s compilation), NetCDF read/write round-trip, <code>open_mfdataset</code> , weighted spatial mean, monthly climatology, zonal mean, albedo calculation, cartopy map rendering, Dask chunked lazy I/O

Result: 90/90 pass on native arm64.

End-to-end simulation validation

Beyond the automated test suite, a full NEON simulation was run inside the container:

1. Pre-download global input data (~6 GB, 31 files) via the `pre-download-inputdata.sh` script with retries and server fallback (GDEX, SVN, FTP)
2. Create a KONZ transient case via `run_tower --neon-sites KONZ --run-type transient`
3. Build the Fortran model (~100 seconds, produces `cesm.exe`)
4. Download NEON tower forcing data (84 monthly files from Google Cloud Storage)
5. Run via `case.submit --no-batch` (CIME manages MPI execution)
6. Archive output (history files, restart files, logs)
7. Read output with xarray: 31 CLM variables, 48 half-hourly time steps

Output includes soil temperature (TSOI), soil moisture (H2OSOI), and sensible heat flux (FSH), which are the variables needed for model-observation comparison.

Known limitations and follow-ups

1. **No CI test integration.** The test suite exists but is not wired into the GitHub Actions workflow. Adding a test job that runs tier0+tier1 on both architectures would be a natural follow-up.
2. **amd64 not tested locally.** All local testing was on native arm64 (Apple Silicon). The amd64 path has not been validated beyond CI builds.
3. **Image size.** At ~7 GB uncompressed, the image is large. The CTSM source tree is the dominant contributor. A production optimization would be to separate build tools into a builder stage, but CTSM's in-tree build model makes this difficult.
4. **Global data not bundled.** The ~6 GB of global input data must be downloaded at runtime. A pre-download script handles this reliably, but initial setup takes 10-15 minutes. Caching on university S3 or bundling in the image would eliminate this cost.
5. **No `--mount=type=cache` for conda.** The Dockerfile does not use BuildKit cache mounts for the conda package cache. Adding this could speed up cache-miss rebuilds.
6. **Multi-day simulation not yet validated.** The 1-day KONZ run is confirmed working. Longer runs (months to years) and additional NEON sites should be validated next.